Fundamentos de C#

Parte 2

Arrays, Coleções, LINQ e mais!

Prof. Geovani

24/02/2026

Marcos Importantes

Data	Evento	Detalhes
03/02	■ Aula 1	Apresentação (Realizada)
17/03	■ Entrega 1	Proposta do Projeto
05/05	■ AP (30%)	Avaliação Parcial
26/05	■ Entrega 2	Modelagem do BD
02/06	■ Aula 16	JWT - ÚLTIMA AULA
09/06	■ AF (30%)	APRESENTAÇÃO FINAL
16/06	■ AI (40%)	Avaliação Interdisciplinar
23/06	■ AIS	Substitutiva (se aplicável)
30/06	■ Aula 20	Encerramento

Atenção ao nosso cronograma

Aula	Data	Conteúdo Principal	Evento/Entrega
1	03/02	Apresentação da Disciplina	
~~2~~	~~10/02~~	~~Fundamentos C# - Parte 1~~	~~-~~
-	17/02	CARNAVAL - Sem aula	Feriado
3	24/02	Fundamentos C# Parte 2 + Coleções	-
4	03/03	OOP - Parte 1	-
5	10/03	OOP - Parte 2	-
6	17/03	.NET Core e ASP.NET Core	Entrega Proposta de Projeto
7	24/03	SOLID + Design Patterns	-
8	31/03	Clean Architecture	-
-	03-04/04	Paixão de Cristo - Suspensão	Feriado
9	07/04	HTTP e REST	-
10	14/04	Construindo APIs REST	-
-	21/04	Tiradentes - Feriado	Feriado
11	28/04	Middleware e Filtros	-
12	05/05	AVALIAÇÃO PARCIAL (AP)	AP - 30%
13	12/05	PostgreSQL + EF Core	-
14	19/05	Repository Pattern + BD	-
15	26/05	Queries Avançadas	Entrega da Arquitetura de Software do projeto
16	**02/06**	**AUTENTICAÇÃO JWT**	**FIM CONTEÚDO**
-	04-06/06	Corpus Christi - Suspensão	Feriado
17	09/06	APRESENTAÇÃO FINAL (AF)	AF - 30%
18	16/06	AVALIAÇÃO INTERDISCIPLINAR (AI)	AI - 40%
19	23/06	Avaliação Substitutiva (AIS)	AIS
20	30/06	Encerramento + Revisão	Fim Semestre

Escopo da Avaliação Formativa

- Requisitos técnicos:
  - No máximo e ao menos 4 entidades relacionais
  - CRUD
  - API REST
  - Autenticação JWT

- Sugestões de temas: disponível no Moodle

Revisão: Aula 2

- História do C# e .NET
- Tipos de dados primitivos
- Variáveis e constantes
- Operadores (aritméticos, lógicos, relacionais)
- Estruturas de controle (if/else, switch)
- Estruturas de repetição (for, while, foreach)
- Exercícios práticos (calculadora, IMC, tabuada)

Objetivos de Hoje

- Trabalhar com Arrays e Arrays Multidimensionais
- Dominar Coleções Genéricas (List, Dictionary, HashSet)
- Manipular Strings eficientemente
- Criar e usar Métodos e Funções
- Entender Tratamento de Exceções
- Introdução ao LINQ (consultas em coleções)

Arrays em C#

```
// Declaração e Inicialização
int[] numeros = new int[5];

// Com valores iniciais
int[] nums = {1, 2, 3, 4, 5};

// Acessando elementos
int primeiro = nums[0];  // 1
int ultimo = nums[4];    // 5

// Tamanho do array
int tamanho = nums.Length; // 5
```

📌 **Características:**

- • Tamanho FIXO
- • Índice inicia em 0
- • Mesmo tipo de dados

Arrays Multidimensionais

```
// Array 2D (Matriz)
int[,] matriz = new int[3, 3];

// Com valores
int[,] mat = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

// Acessar elemento
int valor = mat[1, 2]; // 6
```

```
// Array Jagged (Array de Arrays)
int[][] jagged = new int[3][];
jagged[0] = new int[] {1, 2};
jagged[1] = new int[] {3, 4, 5};
jagged[2] = new int[] {6};

// Tamanhos diferentes!
// jagged[0] tem 2 elementos
// jagged[1] tem 3 elementos
// jagged[2] tem 1 elemento
```

Métodos Úteis: Arrays

Método	Descrição	Exemplo
Length	Tamanho do array	arr.Length
Sort()	Ordena o array	Array.Sort(arr)
Reverse()	Inverte o array	Array.Reverse(arr)
IndexOf()	Busca elemento	Array.IndexOf(arr, 5)

💡 **Arrays são rápidos mas têm tamanho fixo. Para tamanho dinâmico, use List<T>!**

Coleções Genéricas

List<T>, Dictionary<K,V>, HashSet<T>, Queue<T>, Stack<T>

List<T> - Lista Dinâmica

```
// Criar lista
List<string> nomes = new List<string>();

// Adicionar elementos
nomes.Add("João");
nomes.Add("Maria");

// Acessar
string primeiro = nomes[0];

// Remover
nomes.Remove("João");
nomes.RemoveAt(0);
```

📌 **Métodos Principais:**

- • Add() - Adiciona
- • Remove() - Remove por valor
- • RemoveAt() - Remove por índice
- • Find() - Busca elemento
- • Sort() - Ordena
- • Count - Quantidade
- • Clear() - Limpa tudo

✨ **Tamanho DINÂMICO!**

Dictionary<TKey, TValue> - Chave-Valor

```
// Criar dicionário
Dictionary<string, int> idades =
    new Dictionary<string, int>();

// Adicionar
idades["João"] = 25;
idades.Add("Maria", 30);

// Acessar
int idade = idades["João"];

// Verificar se existe
if (idades.ContainsKey("Ana"))
{ ... }
```

📌 **Quando usar:**

- • Busca por CHAVE única
- • Associar valores
- • Performance O(1)

💡 **Exemplo:**

CPF → Pessoa
ID → Produto

HashSet, Queue e Stack

HashSet<T> - SEM DUPLICATAS

```
HashSet<int> nums = new HashSet<int>();
nums.Add(1); nums.Add(1); // Só 1 vez!
```

Queue<T> - FILA (FIFO)

```
Queue<string> fila = new Queue<string>();
fila.Enqueue("A"); // Adiciona
var primeiro = fila.Dequeue(); // Remove
```

Stack<T> - PILHA (LIFO)

```
Stack<int> pilha = new Stack<int>();
pilha.Push(1); // Empilha
var topo = pilha.Pop(); // Desempilha
```

📦 HashSet: Garantir unicidade (emails, IDs)
🚶 Queue: Fila de atendimento, processamento
📚 Stack: Desfazer (Ctrl+Z), navegação (voltar)

Manipulação de Strings

Método	Descrição	Exemplo
Length	Tamanho	texto.Length
ToUpper()	Maiúsculas	texto.ToUpper()
ToLower()	Minúsculas	texto.ToLower()
Substring()	Parte da string	texto.Substring(0, 5)
Replace()	Substituir	texto.Replace("a", "x")
Split()	Dividir	texto.Split(' ')
Trim()	Remove espaços	texto.Trim()
Contains()	Contém?	texto.Contains("abc")
StartsWith()	Começa com?	texto.StartsWith("A")

Interpolação e StringBuilder

String Interpolation

```
string nome = "João";
int idade = 25;

// Forma antiga (concatenação)
string msg1 = "Olá " + nome + "!";

// Forma moderna (interpolação)
string msg2 = $"Olá {nome}!";
string msg3 = $"{nome} tem {idade} anos";
```

StringBuilder

Para concatenações múltiplas

```
StringBuilder sb = new StringBuilder();
sb.Append("Olá");
sb.Append(" ");
sb.Append("Mundo");
string resultado = sb.ToString();
```

💡 **Mais eficiente em loops!**

Métodos e Funções

```
// Método sem retorno (void)
void ImprimirMensagem(string msg)
{
    Console.WriteLine(msg);
}

// Método com retorno
int Somar(int a, int b)
{
    return a + b;
}

// Chamando métodos
ImprimirMensagem("Olá!");
int resultado = Somar(5, 3); // 8
```

Parâmetros Especiais

```
// Parâmetros Opcionais
void Saudar(string nome, string saudacao = "Olá")
{
    Console.WriteLine($"{saudacao}, {nome}!");
}

Saudar("João");            // Olá, João!
Saudar("Maria", "Oi");     // Oi, Maria!

// Parâmetros Nomeados
Saudar(nome: "Pedro", saudacao: "E aí");
```

```
// ref - Passagem por Referência
void Dobrar(ref int numero)
{
    numero *= 2;
}

int x = 5;
Dobrar(ref x);
Console.WriteLine(x); // 10

// out - Saída obrigatória
bool TryParse(string s, out int result)
{ ... }
```

Sobrecarga de Métodos (Overload)

```
// Mesmo nome, assinaturas diferentes

int Somar(int a, int b)
{
    return a + b;
}

double Somar(double a, double b)
{
    return a + b;
}

int Somar(int a, int b, int c)
{
    return a + b + c;
}

// C# escolhe o método correto automaticamente!
```

Tratamento de Exceções

try-catch-finally

try-catch-finally

```
try
{
    // Código que pode gerar erro
    int resultado = 10 / 0;
}
catch (DivideByZeroException ex)
{
    // Trata erro específico
    Console.WriteLine("Não pode dividir por zero!");
}
catch (Exception ex)
{
    // Trata qualquer outro erro
    Console.WriteLine($"Erro: {ex.Message}");
}
finally
{
    // SEMPRE executa (com ou sem erro)
    Console.WriteLine("Finalizando...");
}
```

Tipos Comuns de Exceções

Exceção	Quando ocorre
NullReferenceException	Acesso a objeto null
DivideByZeroException	Divisão por zero
IndexOutOfRangeException	Índice inválido em array
ArgumentException	Argumento inválido
FormatException	Conversão de tipo falhou
InvalidOperationException	Operação inválida
FileNotFoundException	Arquivo não encontrado

💡 **Sempre capture exceções específicas ANTES de Exception genérica!**

Criando Exceções Customizadas

```
// Criar exceção customizada
public class IdadeInvalidaException : Exception
{
    public IdadeInvalidaException(string mensagem)
        : base(mensagem)
    { }
}

// Usar exceção customizada
void ValidarIdade(int idade)
{
    if (idade < 0 || idade > 120)
    {
        throw new IdadeInvalidaException(
            "Idade deve estar entre 0 e 120");
    }
}
```

Introdução ao LINQ

Language Integrated Query

O que é LINQ?

🔍 **LINQ = Language Integrated Query**

- Consultas em coleções (listas, arrays, etc)
- Sintaxe SQL-like em C#
- Filtrar, ordenar, agrupar dados
- Código mais limpo e legível

💡 **Usado MASSIVAMENTE em aplicações .NET!**

LINQ: Where (Filtrar)

```
// Sem LINQ (forma tradicional)
List<int> numeros = new List<int>
    {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

List<int> pares = new List<int>();
foreach (int n in numeros)
{
    if (n % 2 == 0)
        pares.Add(n);
}
```

```
// Com LINQ (moderno)
List<int> numeros = new List<int>
    {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

var pares = numeros
    .Where(n => n % 2 == 0)
    .ToList();

// Resultado: {2, 4, 6, 8, 10}
```

✨ **MUITO mais limpo!**

LINQ: Select (Transformar)

```
List<string> nomes = new List<string> { "João", "Maria", "Pedro" };

// Transformar em maiúsculas
var nomesUpper = nomes.Select(n => n.ToUpper()).ToList();
// Resultado: {"JOÃO", "MARIA", "PEDRO"}

// Transformar em tamanhos
var tamanhos = nomes.Select(n => n.Length).ToList();
// Resultado: {4, 5, 5}

// Combinar Where + Select
var resultado = numeros
    .Where(n => n > 5)
    .Select(n => n * 2)
    .ToList();
```

LINQ: Outros Métodos Importantes

Método	O que faz	Exemplo
OrderBy()	Ordena crescente	.OrderBy(n => n)
OrderByDescending()	Ordena decrescente	.OrderByDescending(n => n)
First()	Primeiro elemento	.First()
FirstOrDefault()	Primeiro ou default	.FirstOrDefault()
Last()	Último elemento	.Last()
Any()	Existe algum?	.Any(n => n > 5)
All()	Todos atendem?	.All(n => n > 0)
Count()	Conta elementos	.Count()
Sum()	Soma valores	.Sum()
Average()	Média	.Average()

Hora de Praticar!

Exercícios com Arrays, Coleções e LINQ

Prática: Agenda de Contatos

Objetivo: Criar sistema de agenda com List<T>

📋 **Requisitos:**

- • Criar classe Contato (Nome, Telefone, Email)
- • Usar List<Contato> para armazenar
- • Menu com opções:

- 1. Adicionar contato
- 2. Listar todos
- 3. Buscar por nome
- 4. Remover contato
- 5. Sair

💡 **Dica: Use LINQ para buscar contatos!**

Fim de Jogo

https://w.app/gil

Obrigado e até a próxima aula!